
CS 61A

Spring 2018

Structure and Interpretation of Computer Programs

MIDTERM 2

INSTRUCTIONS

- You have 2 hours to complete the exam.
- The exam is closed book, closed notes, closed computer, closed calculator, except two hand-written $8.5" \times 11"$ crib sheet of your own creation and the official CS 61A midterm 1 and midterm 2 study guides.
- Mark your answers **on the exam itself**. We will *not* grade answers written on scratch paper.

Last name	
First name	
Student ID number	
CalCentral email (<code>_@berkeley.edu</code>)	
TA	
Name of the person to your left	
Name of the person to your right	
<i>All the work on this exam is my own.</i> (please sign)	

POLICIES & CLARIFICATIONS

- If you need to use the restroom, bring your phone and exam to the front of the room.
- You may use built-in Python functions that do not require import, such as `min`, `max`, `pow`, `len`, and `abs`.
- You **may not** use example functions defined on your study guides unless clearly specified by the question.
- For fill-in-the blank coding problems, we will only grade work written in the provided blanks. You may only write one Python statement per blank line, and it must be indented to the level that the blank is indented.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may use the `Tree`, `Link`, and `BTree` classes defined on Page 2 (left column) of the Midterm 2 Study Guide.

1. (10 points) Buggy Quidditch *(All are in Scope: WWPDP, Object-Oriented Programming)*

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines.

- If an error occurs, write **ERROR**, but include all output displayed before the error.
- To display a function value, write **FUNCTION**.
- If an expression would take forever to evaluate, write **FOREVER**.

The interactive interpreter displays the contents of the `repr` string of the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the code shown on the left first, then you evaluate each expression on the right in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```

1 class Ball:
2     points = 0
3     time = lambda: 'Draco'
4
5     def score(self, who):
6         print(who, self.points)
7
8     def __str__(self):
9         return 'Magic'
10
11 class Snitch(Ball):
12     points = 100
13     time = lambda: 'Harry'
14
15     def __init__(self):
16         self.points = self.points + 50
17
18     def score(self, p):
19         if not time():
20             print(Ball().score(p))
21         else:
22             Ball.score(self, p)
23
24 def chase(r):
25     r.time = Snitch.time
26     r.points += 1
27     quaffle.points += 10
28     print(r().points)
29
30 quaffle = Ball()
31 quaffle.points = 10
32 chasing = quaffle.score
33 time = lambda: Ball.points
34 malfoy = lambda: Ball.time()

```

Expression	Interactive Output
Snitch().points	
chasing(quaffle)	
Snitch().score('Seeker')	
chase(Ball)	
Snitch().score(malfoy())	

3. (12 points) Lists

Definition. A *grid* is a list of lists. Each list in a grid is called a *row*, and all rows must have the same length. `[[1, 2], [3, 4]]` is a grid of integers, but `[[1, 2], [3, 4, 5]]` is not a grid.

- (a) (2 pt) (*All are in Scope: Python Lists*) Implement `column`, which takes a grid `g` and a non-negative integer `c` that is smaller than the length of a row in `g`. It returns a list containing the element at index `c` of each row in the grid.

```
def column(g, c):
    """Return the column of g at index c.

    >>> column([[3, 4, 5], [6, 7, 8], [9, 10, 11]], 1)
    [4, 7, 10]
    """

    return _____
```

- (b) (4 pt) (*All are in Scope: Python Lists*) Implement `print_grid`, which takes a grid `g`. It prints one line for each row in `g`. Line `k` displays each element in row `k`, separated by the minimum number of spaces needed to align the left edges of the `str` strings for the elements of each column of the grid. At least one space should appear between any two elements. Extra spaces at the end of a line are ok. **You may not use the `column` function above. Assume that `g` has at least one row.**

```
def print_grid(g):
    """Print each row on a separate line with columns aligned.

    >>> print_grid([[1, 234, 50, 4, 5], [67, 8, 90, 0, 500], [3, 4, 5, -500, 7]])
    1 234 50 4    5
    67 8   90 0   500
    3 4    5 -500 7
    """

    cs = _____

    widths = [ _____ ([len(str( _____ [ _____ ])) for row in g]) for c in cs]

    for row in g:

        line = ''

        for c in cs:

            s = str(_____ )

            line = line + s + ' ' * (_____ )

        print(line)
```

- (c) (4 pt) (*All are in Scope: Python Lists*) Implement `expand`, which takes a grid `g`, a number of rows `h`, a number of columns `w`, and a fill value. It mutates the contents of `g` so that `g` has at least `h` rows and `w` columns. Any added values are `fill`.

```
def expand(g, h, w, fill):
    """Expand grid g so that it has at least h rows and w columns.

    >>> g = [[1, 2, 3], [40, 50, 60]]

    >>> print_grid(expand(g, 2, 5, 10))
    1  2  3  10 10
    40 50 60 10 10

    >>> print_grid(expand(g, 5, 6, 0))
    1  2  3  10 10 0
    40 50 60 10 10 0
    0  0  0  0  0  0
    0  0  0  0  0  0
    0  0  0  0  0  0

    >>> print_grid(expand(g, 0, 0, 5))    # expand never reduces the dimensions of g.
    1  2  3  10 10 0
    40 50 60 10 10 0
    0  0  0  0  0  0
    0  0  0  0  0  0
    0  0  0  0  0  0
    """

    for row in g:

        row._____()

    for k in _____:

        g._____()

    return g
```

- (d) (2 pt) (*All are in Scope: Asymptotic Notation*) Circle the Θ expression that describes how many new values must be added when a grid with n rows and n columns is expanded to $2 \times n$ rows and $2 \times n$ columns using the `expand` function. Assume that `expand` is implemented correctly.

$\Theta(1)$ $\Theta(\log n)$ $\Theta(n)$ $\Theta(n^2)$ $\Theta(2^n)$ None of these

4. (12 points) Sequences

- (a) (6 pt) (*All are in Scope: Linked Lists*) Implement `stretch`, which takes a `Link` instance `s` with no cycles. It mutates `s` so that, for each position k in the original `s`, the k^{th} element is repeated k times. **You do not need to use the name `i`.**

```
def stretch(s, repeat=0):
    """Replicate the kth element k times, for all k in s.

    >>> a = Link(3, Link(4, Link(5, Link(6))))
    >>> stretch(a)
    >>> print(a)
    <3, 4, 4, 5, 5, 5, 6, 6, 6, 6>
    """
    if _____:

        for i in range(repeat):

            _____ = _____

            _____ = _____

            _____
```

- (b) (6 pt) (*All are in Scope: Tree Recursion*) Implement `combo`, which takes two non-negative integers `a` and `b`. It returns the smallest integer that contains all of the digits of `a` in order, as well as all of the digits of `b` in order.

```
def combo(a, b):
    """Return the smallest integer with all of the digits of a and b (in order).

    >>> combo(531, 432)      # 45312 contains both _531_ and 4_3_2.
    45312
    >>> combo(531, 4321)     # 45321 contains both _53_1 and 4_321.
    45321
    >>> combo(1234, 9123)    # 91234 contains both _1234 and 9123_.
    91234
    >>> combo(0, 321)        # The number 0 has no digits, so 0 is not in the result.
    321
    """
    if _____:

        return a + b

    elif _____:

        return combo(_____, _____)

    return _____(_____, _____)

    _____)
```

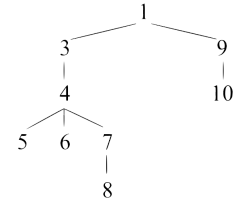
5. (10 points) Trees

Definition. A *sibling* of a node in a tree is another node with the same parent.

- (a) (4 pt) (*All are in Scope: Trees*) Implement `siblings`, which takes a `Tree` instance `t`. It returns a list of the labels of all nodes in `t` that have a sibling. These labels can appear in any order.

```
def siblings(t):
    """Return a list of the labels of all nodes that have siblings in t.

    >>> a = Tree(4, [Tree(5), Tree(6), Tree(7, [Tree(8)])])
    >>> siblings(Tree(1, [Tree(3, [a]), Tree(9, [Tree(10)])]))
    [3, 9, 5, 6, 7]
    """
```



```
result = [_____]
```

```
for b in t.branches:
```

```
_____
```

```
return result
```

- (b) (6 pt) (*All are in Scope: Object-Oriented Programming, Trees*) Implement the `Sib` class that inherits from `Tree`. In addition to `label` and `branches`, a `Sib` instance `t` has an attribute `siblings` that stores the number of siblings `t` has in `Sib` trees containing `t` as a node. **Assume that the branches of a `Sib` instance will never be mutated or re-assigned.**

```
class Sib(Tree):
    """A tree that knows how many siblings it has.

    >>> a = Sib(4, [Sib(5), Sib(6), Sib(7, [Sib(8)])])
    >>> a.label
    4
    >>> a.branches[1].label
    6
    >>> a.siblings
    0
    >>> a.branches[1].siblings
    2
    """
    def __init__(self, label, branches=[]):
```

```
self.siblings = _____
```

```
_____:
```

```
_____
```

```
Tree._____
```